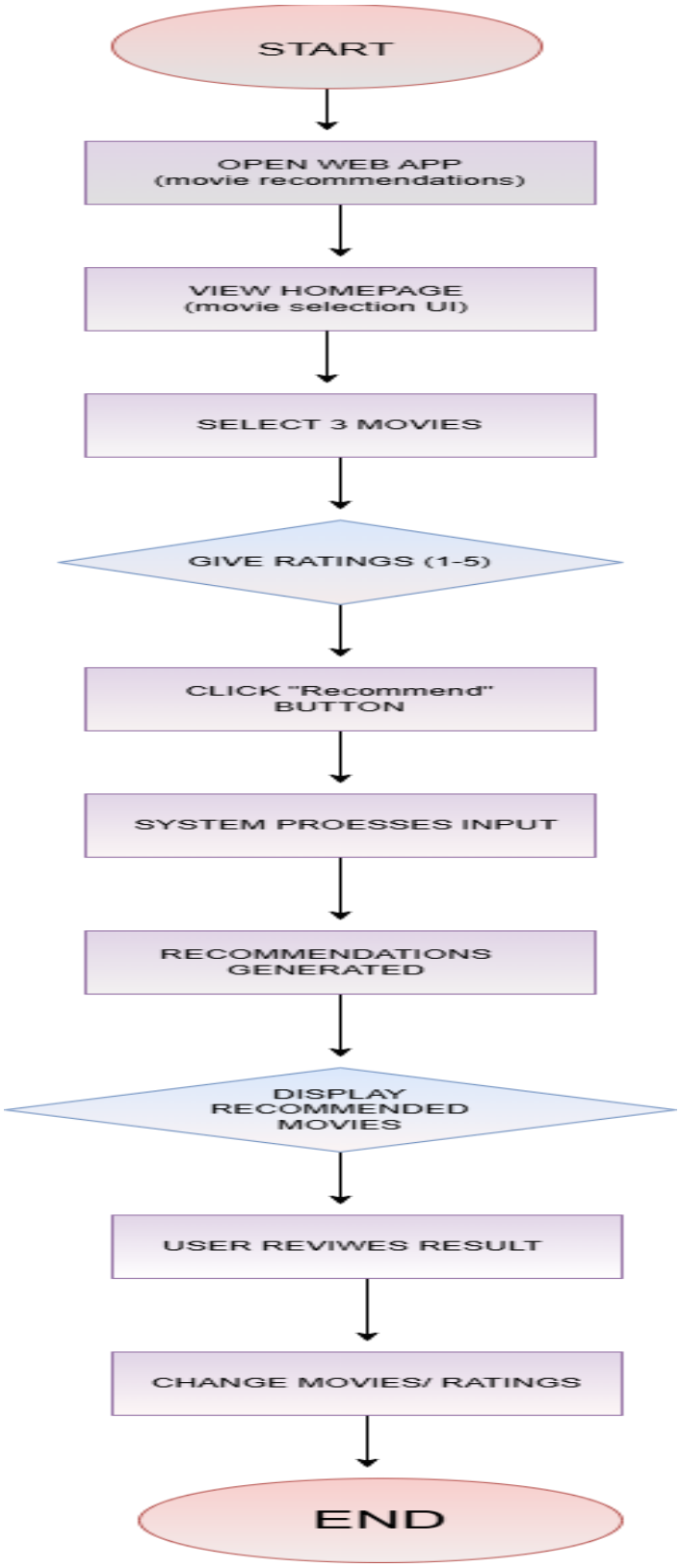
MOVIE RECOMMENDATION SYSTEM

FLOWCHART:



CODE:

```
import React, { createContext, useContext, useState, ReactNode } from "react";

import type { Movie, MovieSelection, RecommendedMovie } from
"@/lib/recommendation";


interface MovieContextType {

  movies: Movie[];

  setMovies: (movies: Movie[]) => void;

  selections: MovieSelection[];

  setSelections: (selections: MovieSelection[]) => void;

  recommendations: RecommendedMovie[];

  setRecommendations: (recs: RecommendedMovie[]) => void;

}


const MovieContext = createContext<MovieContextType | undefined>(undefined);


export const MovieProvider = ({ children }: { children: ReactNode }) => {

  const [movies, setMovies] = useState<Movie[]>([]);

  const [selections, setSelections] = useState<MovieSelection[]>([]);

  const [recommendations, setRecommendations] = useState<RecommendedMovie[]>([]);


  return (

    <MovieContext.Provider

      value={{ movies, setMovies, selections, setSelections, recommendations,
setRecommendations }}

    >

      {children}
```

```
    </MovieContext.Provider>
  );
};

export const useMovies = () => {
  const ctx = useContext(MovieContext);
  if (!ctx) throw new Error("useMovies must be used within MovieProvider");
  return ctx;
};
export interface Movie {
  title: string;
  overview: string;
  genres: string;
  language: string;
  vote_average: number;
}

export interface MovieSelection {
  movie: Movie;
  userRating: number;
}

export interface RecommendedMovie extends Movie {
  score: number;
}

function tokenize(text: string): string[] {
  return text
```

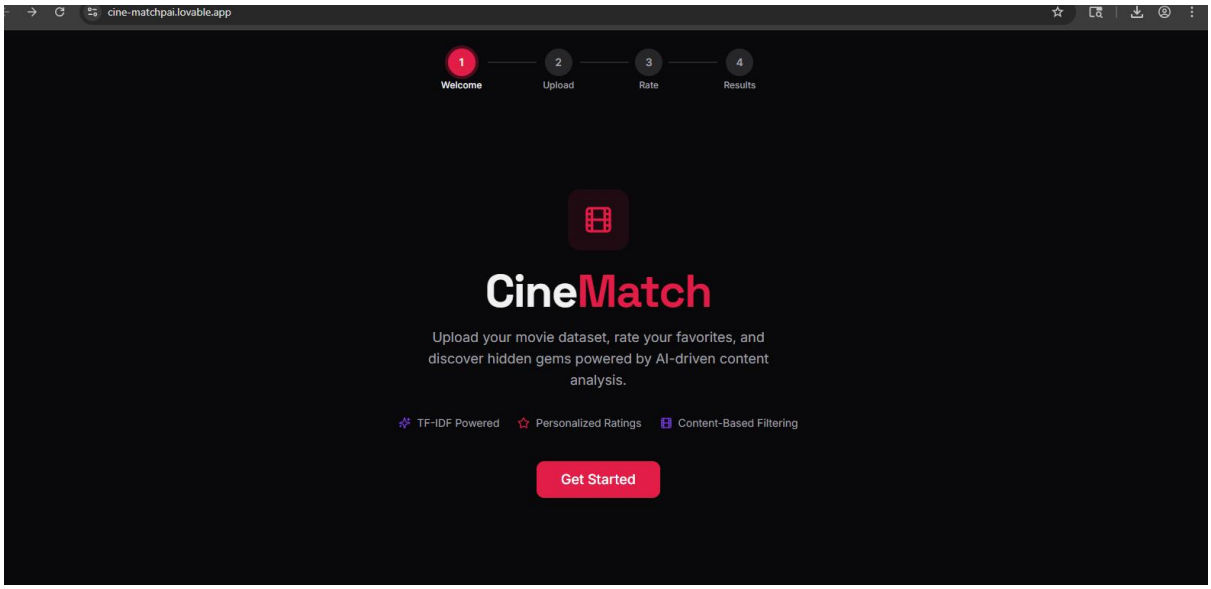
```
    .toLowerCase()
    .replace(/[^a-z0-9\s]/g, "")
    .split(/\s+/)
    .filter((w) => w.length > 2);
}

function computeTfIdf(docs: string[][]): { matrix: number[][]; vocab: string[] } {
  const df: Record<string, number> = {};
  const vocabSet = new Set<string>();

  for (const doc of docs) {
    const unique = new Set(doc);
    for (const word of unique) {
      df[word] = (df[word] || 0) + 1;
      vocabSet.add(word);
    }
  }

  const vocab = Array.from(vocabSet);
  const vocabIndex: Record<string, number> = {};
  vocab.forEach((w, i) => (vocabIndex[w] = i));

  const N = docs.length;
  const matrix: number[][] = [];

  for (const doc of docs) {
    const tf: Record<string, number> = {};
    for (const word of doc) {
```

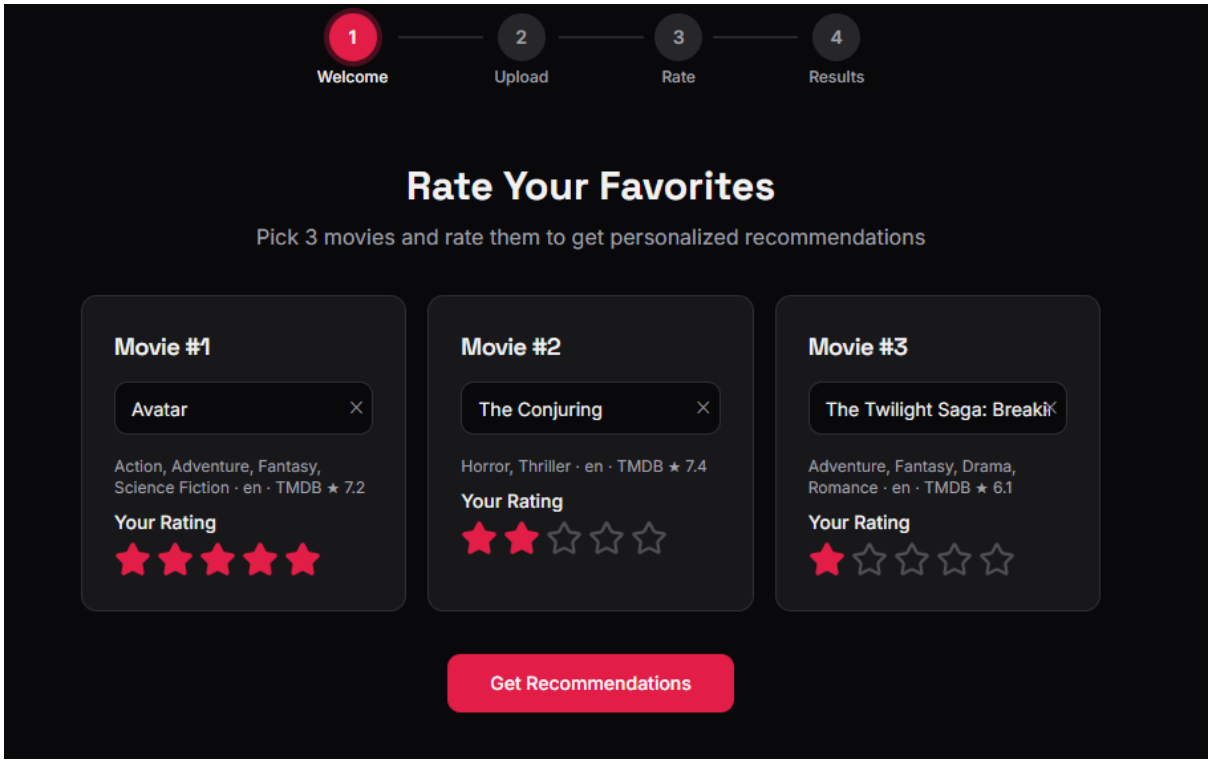
```
      tf[word] = (tf[word] || 0) + 1;
    }
    const row = new Array(vocab.length).fill(0);
    for (const word of Object.keys(tf)) {
      const idx = vocabIndex[word];
      if (idx !== undefined) {
        row[idx] = (tf[word] / doc.length) * Math.log(N / (df[word] || 1));
      }
    }
    matrix.push(row);
  }

  return { matrix, vocab };
}

function cosineSim(a: number[], b: number[]): number {
  let dot = 0, magA = 0, magB = 0;
  for (let i = 0; i < a.length; i++) {
    dot += a[i] * b[i];
    magA += a[i] * a[i];
    magB += b[i] * b[i];
  }
  const denom = Math.sqrt(magA) * Math.sqrt(magB);
  return denom === 0 ? 0 : dot / denom;
}

export function getRecommendations(
  movies: Movie[],
```

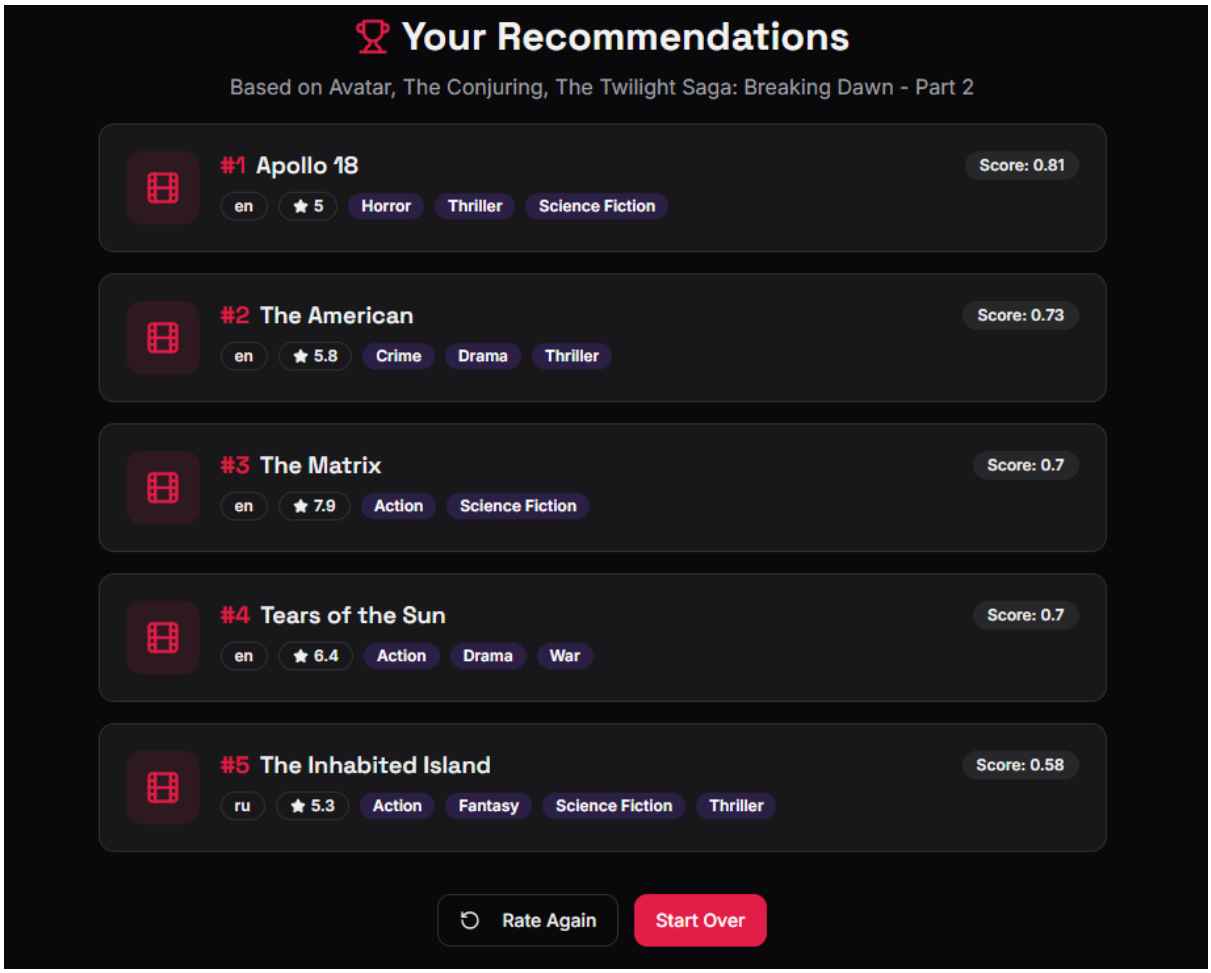
```
  selections: MovieSelection[]
): RecommendedMovie[] {
  const tokenized = movies.map((m) => tokenize(m.overview || ""));
  const { matrix } = computeTfIdf(tokenized);

  const selectedTitles = new Set(selections.map((s) => s.movie.title));
  const scores = new Array(movies.length).fill(0);

  for (const sel of selections) {
    const selIdx = movies.findIndex((m) => m.title === sel.movie.title);
    if (selIdx === -1) continue;

    for (let i = 0; i < movies.length; i++) {
      if (selectedTitles.has(movies[i].title)) continue;
      const sim = cosineSim(matrix[selIdx], matrix[i]);
      scores[i] += sim * sel.userRating;
    }
  }

  const indexed = movies
    .map((m, i) => ({ movie: m, score: scores[i] }))
    .filter((x) => !selectedTitles.has(x.movie.title) && x.score > 0)
    .sort((a, b) => b.score - a.score)
    .slice(0, 5);

  return indexed.map((x) => ({ ...x.movie, score: Math.round(x.score * 100) / 100 }));
}
import { clsx, type ClassValue } from "clsx";
```

```
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

OUTPUT:




1
Welcome
2
Upload
3
Rate
4
Results
Rate Your Favorites
Pick 3 movies and rate them to get personalized recommendations
Movie #1
Avatar
Action, Adventure, Fantasy, Science Fiction · en · TMDB ★ 7.2
Your Rating
Movie #2
The Conjuring
Horror, Thriller · en · TMDB ★ 7.4
Your Rating
Movie #3
The Twilight Saga: Breakin
Adventure, Fantasy, Drama, Romance · en · TMDB ★ 6.1
Your Rating
Get Recommendations


Your Recommendations
Based on Avatar, The Conjuring, The Twilight Saga: Breaking Dawn - Part 2
#1 Apollo 18
Score: 0.81
en
5
Horror
Thriller
Science Fiction
#2 The American
Score: 0.73
en
5.8
Crime
Drama
Thriller
#3 The Matrix
Score: 0.7
en
7.9
Action
Science Fiction
#4 Tears of the Sun
Score: 0.7
en
6.4
Action
Drama
War
#5 The Inhabited Island
Score: 0.58
ru
5.3
Action
Fantasy
Science Fiction
Thriller
Rate Again
Start Over

BY:

160124748014 MAHANYA

160124748018 YASHNI